Suíte de testes e análise crítica

Sprint 3 - Parte 1 | Frontend em modo demonstração

1. Contexto e escopo

O sistema permite ao professor abrir uma chamada com código temporário e ao aluno registrar presença. Foram selecionadas cinco histórias de trabalho: abertura, registro, frequência, tempo restante e persistência. Os rótulos US01-US05 foram criados nesta atividade; a equipe ainda deve reconciliá-los com o backlog da Sprint 2. A persistência (US05) é suporte técnico ao fluxo.

Stack: React 18, TypeScript 5.9 e Vite 7. Ferramentas: Vitest 3.2.4 e @vitest/coverage-v8 3.2.4, ambiente Node. Geração: ChatGPT/Codex, família GPT-6 informada pelo ambiente, em 11/11/2026; snapshot interno não exposto. Não se inventou número de versão do modelo.

A medição inclui attendance.ts e summary.ts, que concentravam as regras e a persistência antes das três novas refatorações. Não mede o backend, autenticação, componentes React nem toda a aplicação.

2. Projeto dos casos e limite do protocolo

A matriz inicial contém **14 critérios, 32 partições e 18 pontos-limite**. Exemplo: duração 1 e 120 aceita; 0 e 121 rejeitada. O registro é válido 1 ms antes de expiresAt e inválido no instante de expiração e 1 ms depois. CT24/CT25 verificam fronteiras de segundo, minuto e zero.

Desvio importante: a matriz foi escrita pelo assistente e commitada antes da geração dos testes, mas não foi elaborada manualmente pela equipe. A mesma IA teve acesso ao contexto da matriz; não houve experimento cego. Portanto este trabalho não comprova o protocolo manual/independente exigido. A equipe deve revisar os critérios e consultar a docente sobre a aceitação ou repetição dessa etapa.

Os critérios foram reconstruídos do escopo acordado e de contratos atuais. As escolhas de 1-120 minutos, arredondamento e tratamento de erro de armazenamento precisam de ratificação. Essa limitação impede tratar cobertura de requisitos como validação independente de todos os requisitos originais.

3. Geração e execução

A v1 foi produzida em uma rodada e preservada sem alterações: 44 casos executaram e passaram, sem erro de carregamento. A resposta integral, instrução de geração registrada pelo assistente, contexto e SHA-256 estão em docs/testes. Não houve rodadas adicionais para corrigir imports. A v2 copia os 44 casos e acrescenta seis casos para lacunas observadas.

Vitest transpila os testes; ausência de erro de carregamento não é prova de tipagem. Um comando separado, npm run check, verifica os tipos do app e dos testes. O build é outra verificação separada.

4. Indicadores no mesmo código

Indicador	v1	v2
Gerados / passaram / falharam / não carregaram	44 / 44 / 0 / 0	50 / 50 / 0 / 0
Linhas executadas	100%	100%
Branches executados	97,67%	100%
Critérios com ao menos um teste	13/14 = 92,86%	14/14 = 100%
Partições exercitadas	27/32 = 84,38%	32/32 = 100%
Pontos-limite exercitados	18/18 = 100%	18/18 = 100%
Sem asserção ou com apenas asserção trivial	0	0
Defeitos reais demonstrados por falha	0	0

Fontes: docs/testes/relatorios/v1/index.html e v2/index.html, seus coverage-summary.json, saídas v1-execucao.txt/v2-execucao.txt e matriz-final.md. As contagens funcionais vêm da inspeção dos testes e da matriz, não do V8. Cada critério conta se tem ao menos um cenário; isso não garante todas as combinações.

Os relatórios v1 e v2 foram medidos antes de DT-01/02/03, com os mesmos arquivos e configuração. A medição pós-refatoração está separada em docs/qualidade/evidencias/cobertura-final. Mover arquivos altera denominadores; não misturamos essa medição com a comparação inicial.

5. Análise crítica com evidências

Técnicas funcionais. A v1 cobriu os 18 limites previstos, incluindo 0/1 e 120/121 minutos e o instante de expiração. Deixou cinco partições sem cenários controlados: CT07 (colisão), CT29 (sem locks), CT30 (JSON inválido), CT31 (callback falhando) e CT32 (falha ao salvar). Não é possível afirmar que a LLM encontrou as mesmas partições de uma equipe independente, pois essa comparação humana não aconteceu.

Requisitos. US05-CA3 não tinha teste na v1. US01-CA3 já contava como coberto pelo formato e unicidade observada, mas faltava provocar uma colisão. Isso mostra a diferença entre 92,86% de critérios e 84,38% de partições. As duas suítes usam testes associados aos critérios desta matriz; a validade desses critérios ainda requer revisão humana.

Linhas versus branches. A v1 executou 100% das linhas, mas deixou um ramo sem execução na linha 47 da versão original de attendance.ts:

```
return navigator.locks
  ? navigator.locks.request(storageKey, save)
  : Promise.resolve().then(save)
```

CT28 percorreu o caminho com lock e verificou que leitura e escrita aconteciam enquanto o lock estava ativo. A mesma expressão ocupa uma linha coberta, mas o fallback não foi executado. CT29 cobre o outro caminho e verifica o conteúdo salvo e devolvido. O relatório V8 passa de 97,67% para 100% de branches, ganho de 2,33 pontos percentuais.

Oráculos. Os testes comparam expiração exata, presença por aluno, rejeições, imutabilidade e percentuais com valores explícitos; a inspeção não encontrou teste sem asserção nem asserção apenas de existência. Entretanto quatro casos da v2 (CT30, duas variantes de CT31, CT32) caracterizam propagação de erro atual, não um critério aprovado independentemente pela equipe. Não foi observado teste passando e validando um defeito, mas isso não é prova de ausência de defeitos.

Risco conhecido: JSON válido com estrutura errada não é validado por readCalls (DT-07). Locks simulados não provam comportamento concorrente real. Mensagens corretas e testes passando não demonstram autorização segura de alunos ou professores.

6. Melhorias da v2

Lacuna	Caso acrescentado	Efeito
Colisão de código	CT07: gerador retorna ABC123 e depois 000102	Confirma repetição do sorteio e preservação do histórico
Fallback	CT29: navigator sem locks	Exercita branch ausente e verifica persistência
JSON inválido	CT30: leitura de {invalid	Caracteriza erro sem silenciosamente substituir dados
Falha de domínio	CT31 com lock e sem lock	Duas variantes; não chama setItem e mantém registros
Falha de gravação	CT32: setItem lança erro	Promise rejeita, sem sucesso falso

A v2 acrescentou seis casos para cinco partições e um critério ausente. Todos os limites já estavam na v1. As metas quantitativas desta matriz foram atingidas: 14/14 critérios, 32/32 partições, 18/18 limites e branches acima da v1. A suíte passou antes e após cada uma das três refatorações, sem mudar os testes para acomodar os novos arquivos.

7. Conclusão e próximos passos

A LLM foi útil para acelerar casos repetitivos e explicitar fronteiras; a medição identificou um caminho estrutural ausente e a matriz expôs lacunas adicionais que 100% de linhas escondia. Usaríamos essa abordagem com critérios aprovados previamente, matriz humana independente, revisão de oráculos e preservação da resposta original. Uma suíte ampla continua sendo uma evidência parcial do comportamento.

Pendências humanas: validar histórias/contratos e matriz, revisar todos os oráculos, registrar achados de inspeção dos integrantes, distribuir os itens no backlog e discutir com a docente o desvio da etapa manual/cega. Não se deve alterar datas ou atribuir a alunos um trabalho que foi produzido pela IA.

Apêndices no repositório

docs/testes/matriz-inicial.md e matriz-final.md; llm-log.md; respostas/v1-resposta-integral.ts.txt e v2-resposta-integral.ts.txt; contexto/; indicadores.json; integridade-v1.sha256. Os testes ficam em p07-frontend/tests/v1-llm e tests/v2. O manual explica como reproduzir as medições históricas sem sobrescrevê-las.